

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

Name: [DIVYADHARSHINI R M]

Roll Number: [24011101028]

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks Performed

1. Downloaded and understood the dataset.
2. Performed Exploratory Data Analysis (EDA).
3. Preprocessed the dataset.
4. Implemented a Single Layer Perceptron from scratch.
5. Trained the model using the perceptron learning rule.
6. Evaluated the classifier using standard performance metrics.
7. Analyzed the effect of different learning rates.
8. Compared the implementation with Scikit-learn's Perceptron.

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

The dataset was loaded, the first five samples inspected, dimensions and missing values checked, and descriptive statistics computed. The dataset contains 1372 samples with no missing values across all 4 features.

Task 2: Exploratory Data Analysis

Histograms, correlation heatmap, scatter plot, and boxplots were generated (see Section 8).

Task 3: Data Preprocessing

All features were normalized using `StandardScaler`, and the dataset was split into 1097 training samples (80%) and 275 testing samples (20%), stratified by class.

Task 4: Perceptron Implementation

A perceptron was implemented from scratch, including weight initialization, bias initialization, step activation function, forward propagation, and the perceptron learning rule.

Task 5: Model Training

The model was trained for 50 epochs at $\eta = 0.01$, with misclassified samples, updated weights, and updated bias recorded at every epoch (see Section 8, Epoch-wise Learning).

Task 6: Model Evaluation

Accuracy, Precision, Recall, F1-score, and Confusion Matrix were computed on the held-out test set (see Section 9).

7. Source Code

```
1 class Perceptron:
2     def __init__(self, n_features, lr=0.01, epochs=50, seed=42):
3         rng = np.random.default_rng(seed)
4         self.w = rng.normal(0, 0.01, n_features)
5         self.b = 0.0
6         self.lr = lr
7         self.epochs = epochs
8
9     def step(self, z):
10        return np.where(z >= 0, 1, 0)
11
12    def predict(self, X):
13        return self.step(X @ self.w + self.b)
14
15    def fit(self, X, y):
16        for epoch in range(1, self.epochs + 1):
17            errors = 0
18            for xi, target in zip(X, y):
19                z = np.dot(xi, self.w) + self.b
20                y_hat = self.step(z)
21                update = self.lr * (target - y_hat)
22                if update != 0:
23                    self.w += update * xi
24                    self.b += update
25            errors += 1
```

```

26         if errors == 0:
27             break

```

Full source code (including EDA, preprocessing, evaluation, learning-rate comparison, and Scikit-learn comparison) is available in the GitHub repository (see References).

8. Mandatory Plots with Interpretation

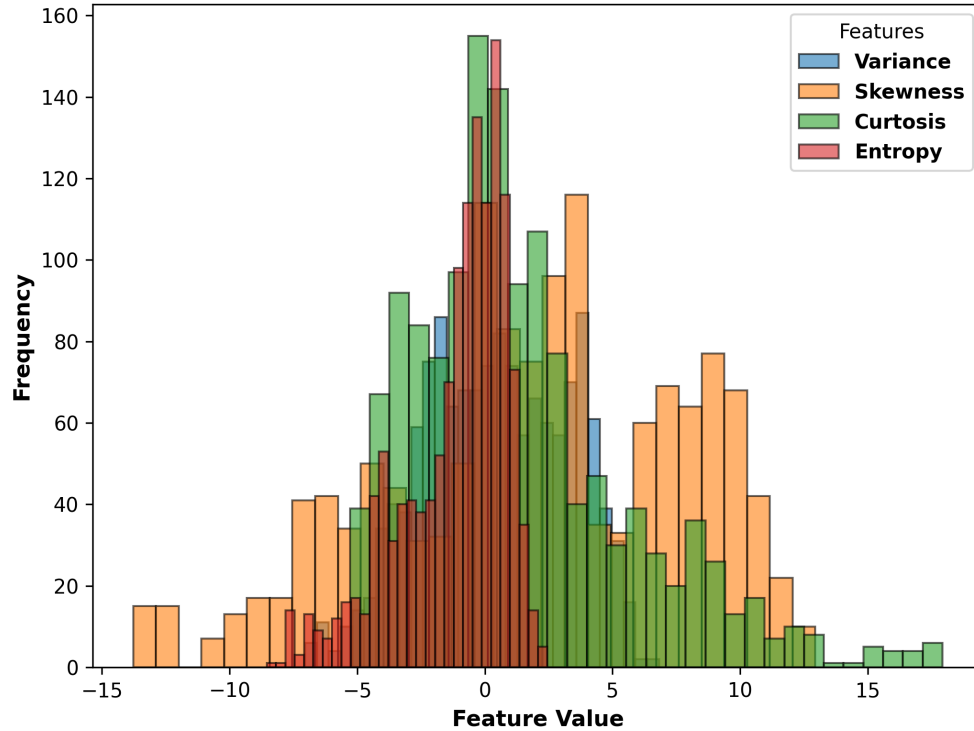


Fig: Combined histogram of all four features, showing overlapping distributions.

Figure 1: Combined Feature Histograms

Combined histogram of all four features, showing overlapping distributions. Entropy and Kurtosis show visible skewness, motivating feature normalization before training.

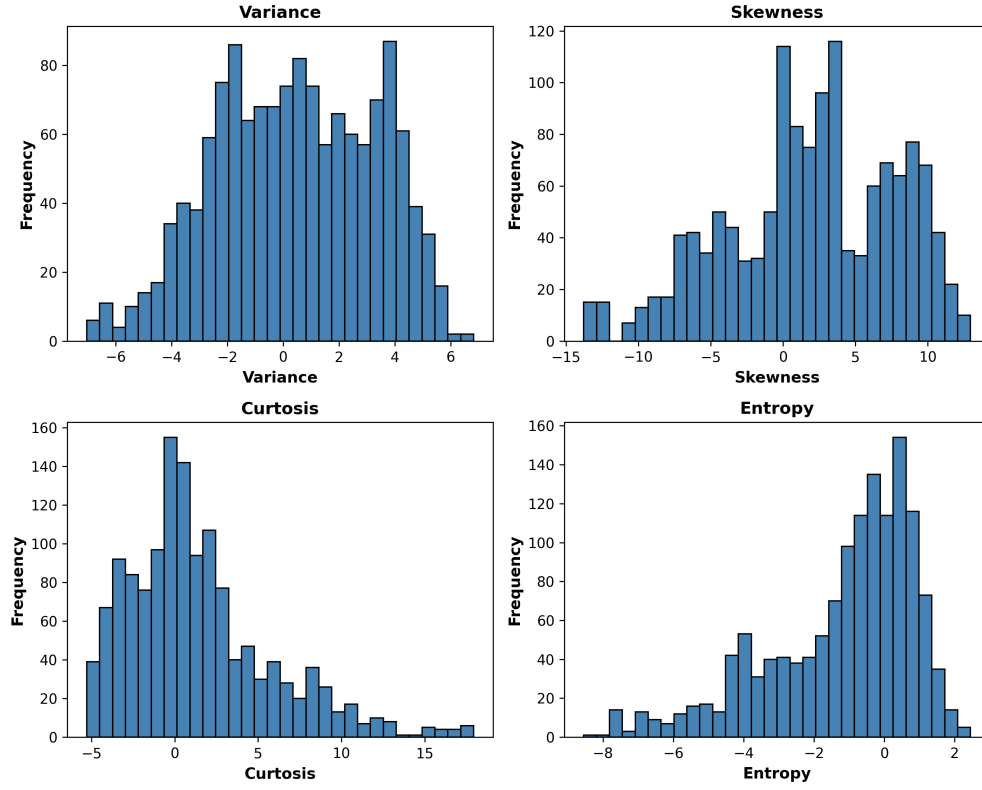


Fig: Individual feature histograms revealing per-feature skewness and outliers.

Figure 2: Per-Feature Histograms

Individual feature histograms reveal Entropy is left-skewed and Kurtosis is right-skewed, while Variance and Skewness are closer to symmetric, confirming non-uniform feature distributions.

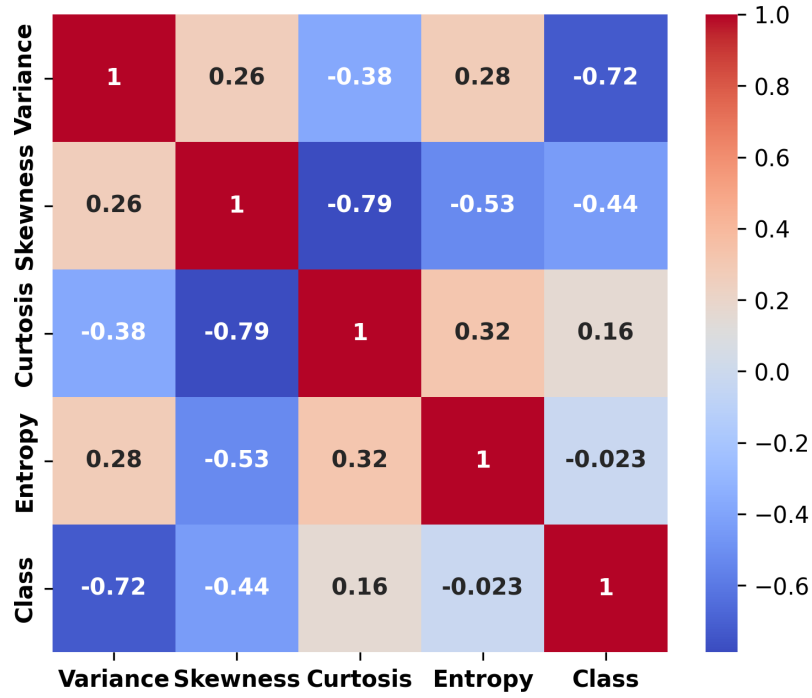


Fig: Correlation heatmap showing pairwise linear relationships between features.

Figure 3: Correlation Heatmap

The heatmap shows Variance and Skewness are the most correlated with the target class, while Entropy shows the weakest correlation, consistent with its near-zero learned weight.

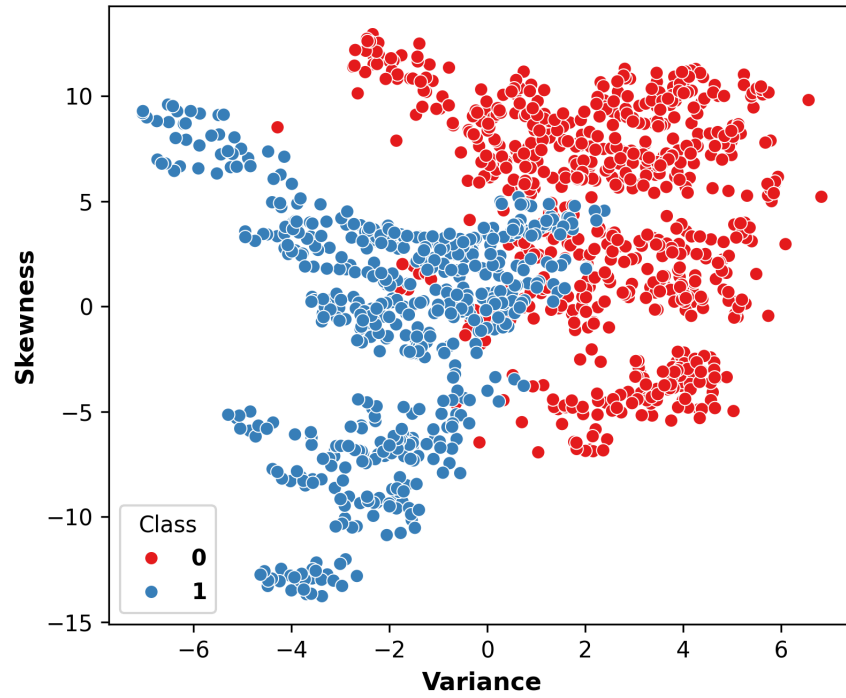


Fig: Scatter plot of Variance vs Skewness, colored by class label.

Figure 4: Scatter Plot: Variance vs Skewness

The scatter plot shows the two classes are approximately linearly separable using just Variance and Skewness, explaining the perceptron's high test accuracy.

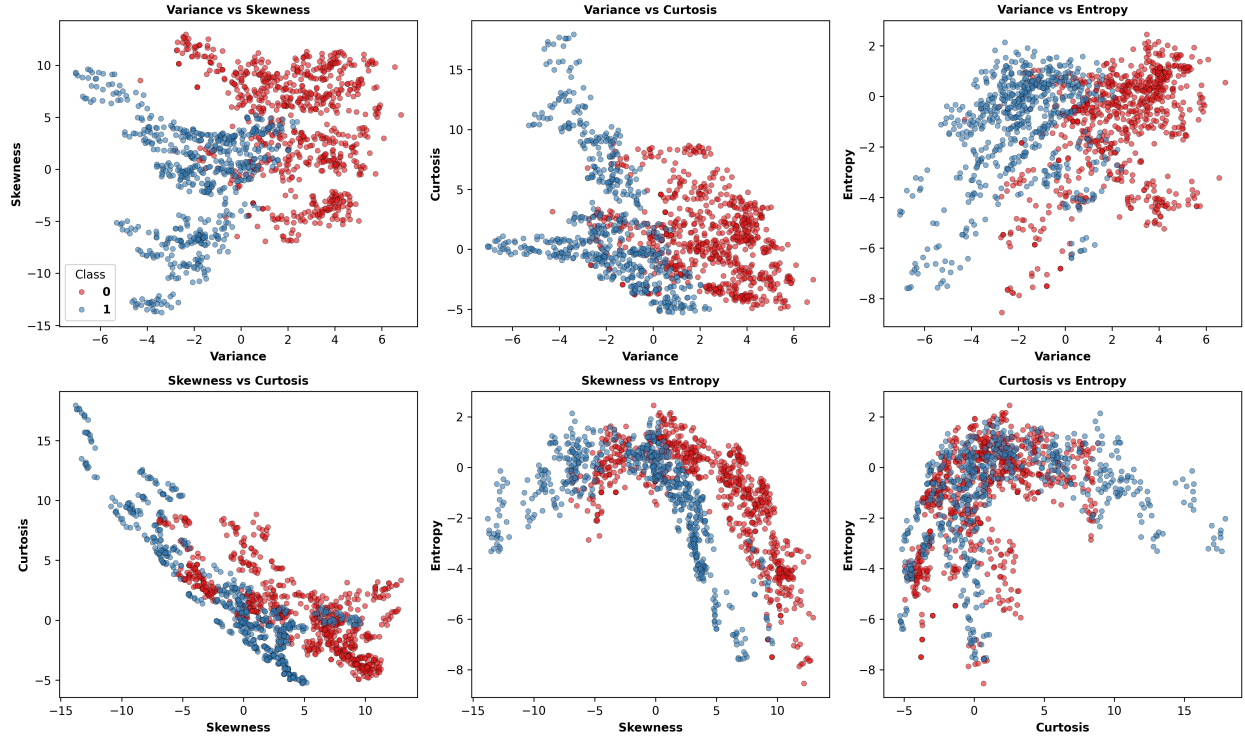


Fig: Scatter plots for all six pairwise feature combinations, colored by class.

Figure 5: Scatter Plots for All Six Pairwise Feature Combinations

Variance-Skewness and Variance-Kurtosis show the clearest class separation, consistent with the correlation heatmap. Skewness-Entropy and Kurtosis-Entropy show more overlap between classes, matching Entropy's near-zero learned weight (w_4) in the trained perceptron, indicating it contributes least to the decision boundary.

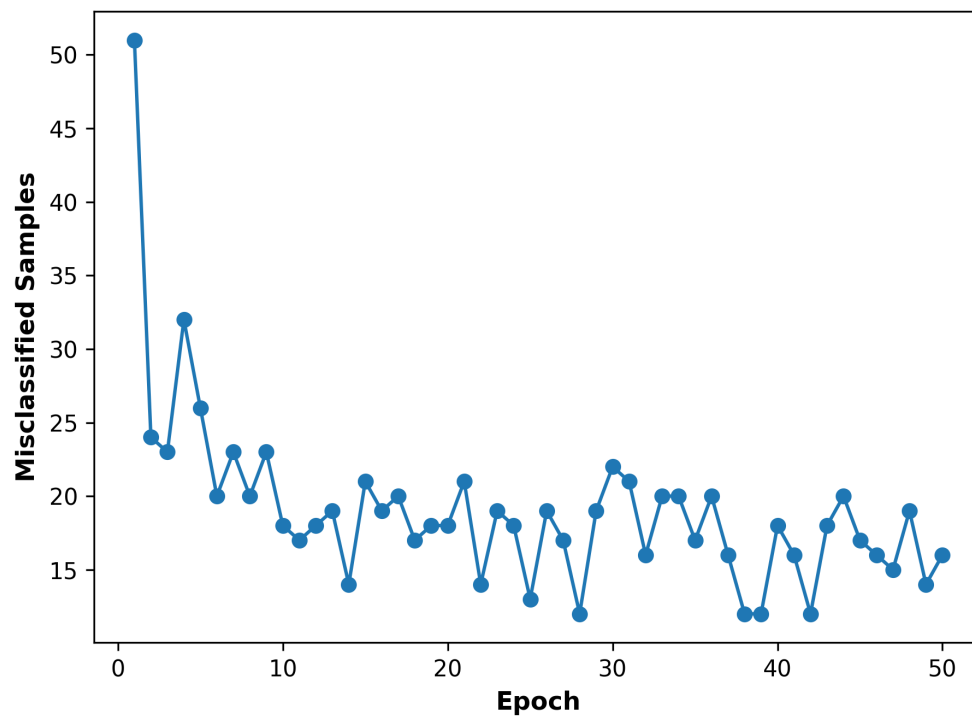


Fig: Training error vs epoch, verifying convergence of the perceptron.

Figure 6: Training Error versus Epoch

Training error drops sharply within the first 5 epochs, then oscillates in a narrow band (15–20 errors) rather than reaching zero, indicating the classes are near- but not perfectly linearly separable.

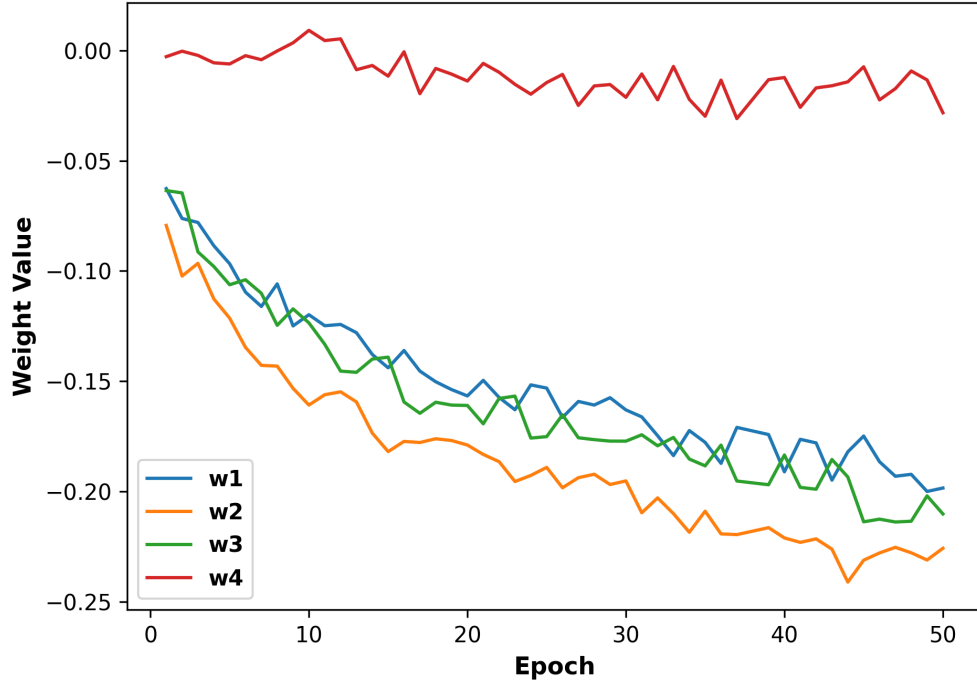


Fig: Weight evolution across epochs, showing stabilization over training.

Figure 7: Weight Evolution across Epochs

Weights w_1 – w_3 (Variance, Skewness, Curtosis) trend negative and stabilize over training, while w_4 (Entropy) remains near zero, indicating it contributes little to the decision boundary.

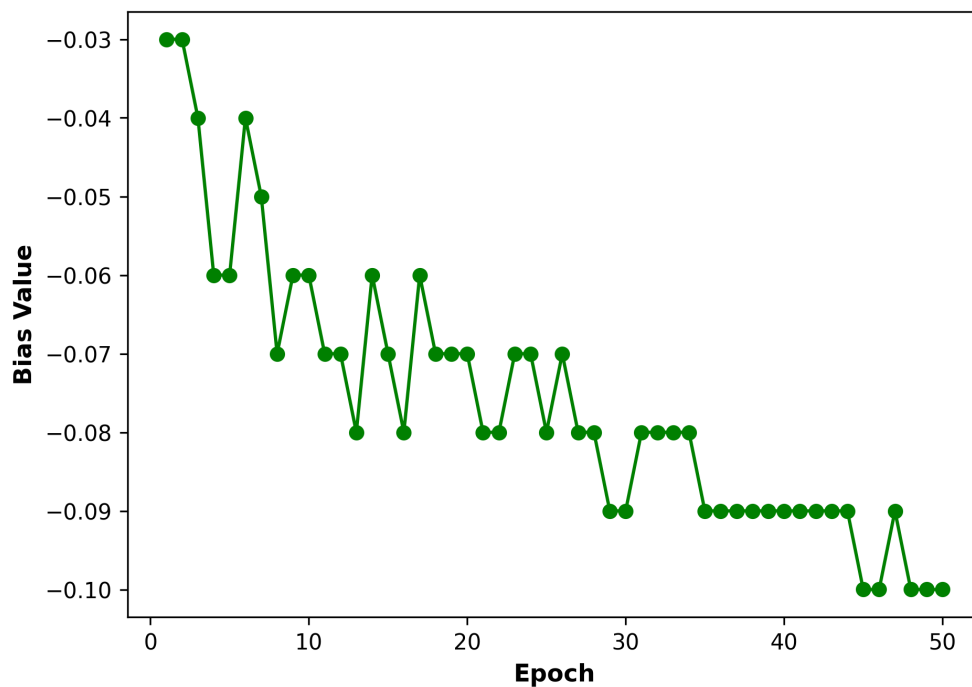


Fig: Bias evolution across epochs, showing the role of bias in convergence.

Figure 8: Bias Evolution across Epochs

The bias steadily decreases and plateaus after approximately epoch 35, showing the decision threshold converging as training progresses.

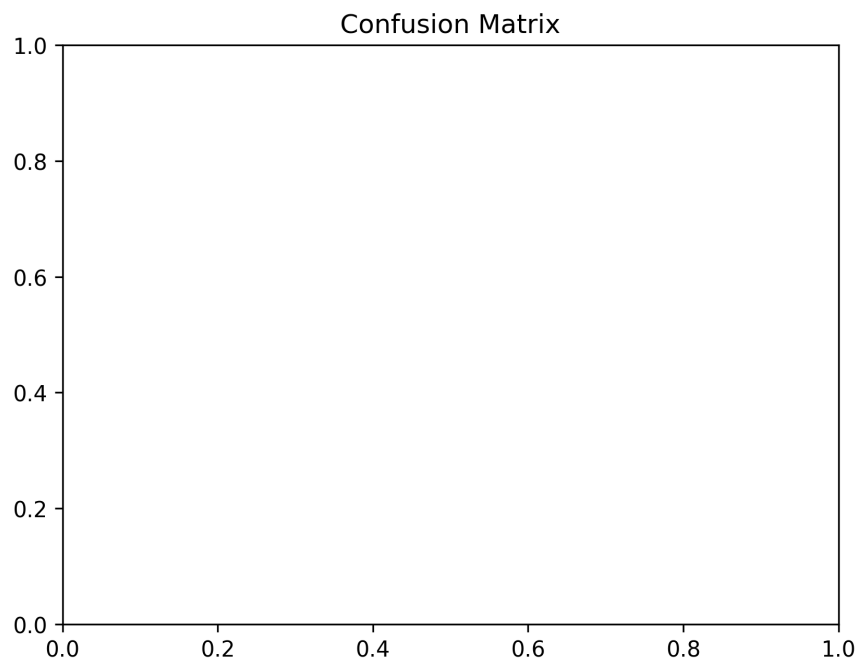


Figure 9: Confusion Matrix

The confusion matrix shows 151 true Authentic, 121 true Forged, 2 false positives, and 1 false negative out of 275 test samples, corresponding to the high precision and recall obtained.

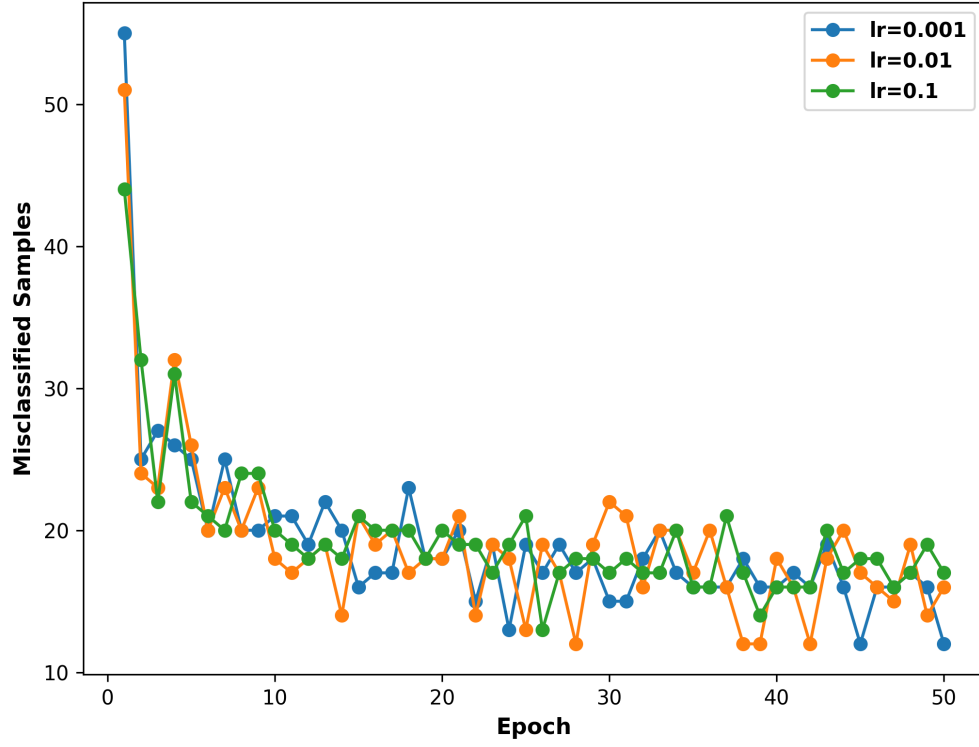


Fig: Comparison of convergence behaviour across different learning rates.

Figure 10: Learning Rate Comparison

All three learning rates (0.001, 0.01, 0.1) converge to a similar error band; $lr=0.001$ is smoother but slower, while $lr=0.1$ is noisier, illustrating the classic stability-speed tradeoff. $lr=0.01$ was selected as the best balance, achieving the highest test accuracy while remaining reasonably stable.

Figure 11: Decision Boundary (Variance vs Skewness)

The decision boundary illustrates the linear hyperplane learned by the perceptron using two selected features, visually confirming near-linear separability of the two classes.

9. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	1097 / 275 (80/20, stratified)
Learning Rate	0.01
Epochs	50
Final Weights	[-0.1985, -0.2258, -0.2102, -0.0282]
Final Bias	-0.10
Accuracy	0.9891
Precision	0.9837
Recall	0.9918
F1-score	0.9878

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	51	-0.0626	-0.0793	-0.03
49	14	-0.2000	-0.2311	-0.10
50	16	-0.1985	-0.2258	-0.10

10. Additional Tasks

1. Step vs Sigmoid Activation: The Step function is non-differentiable, making it unusable with gradient-based optimization (backpropagation) used in modern deep learning. Sigmoid is smooth and differentiable, enabling gradient computation, which is why modern networks use Sigmoid (or other differentiable functions) instead of Step.

2. Comparison with Scikit-learn: The scratch implementation achieved higher test accuracy (0.9891) than Scikit-learn's `Perceptron` (0.9745). This is because Scikit-learn applies early stopping (`tol=1e-3`) and shuffles data every epoch by default, while the scratch model trained the full 50-epoch budget on a fixed sample order, allowing it to reach a marginally better boundary on this dataset.

3. Learning Rate Comparison (0.001, 0.01, 0.1): All three rates converged to a similar error band. `lr=0.01` gave the best test accuracy (0.9891) and F1-score (0.9878) while remaining stable, and was therefore selected as the final learning rate.

4. XOR Limitation: A Single Layer Perceptron cannot solve XOR because XOR outputs are not linearly separable – no single straight line (or hyperplane) can separate the two classes. Solving XOR requires a multilayer network with at least one hidden layer.

5. Effect of Normalization: Feature normalization (via `StandardScaler`) equalized feature scales, preventing features with larger numeric ranges from dominating weight updates, and improved convergence stability during training.

Additional Experiment: Logical Gate Implementation using Perceptron

To further analyze the capabilities and limitations of the Single Layer Perceptron, the model was tested on basic logical gates: AND, OR, NOT, and XOR. These experiments demonstrate the working of the perceptron learning algorithm and highlight the importance of linear separability.

AND Gate

The AND gate was implemented using a Single Layer Perceptron. The model successfully learned the relationship between the two input variables and the output. Since the AND gate is linearly separable, a single decision boundary is sufficient to separate the two classes.

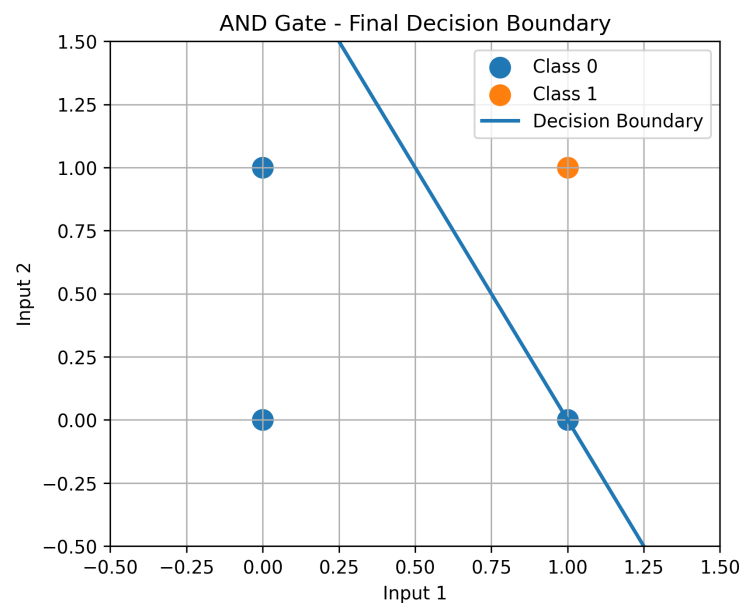


Figure 12: Decision Boundary Learned for AND Gate

The obtained decision boundary correctly classifies the input combinations. This confirms that the perceptron can learn simple logical functions when the data points can be separated using a straight line.

OR Gate

The OR gate was also successfully implemented using the perceptron learning algorithm. Similar to the AND gate, the OR gate is linearly separable, allowing the perceptron to converge to suitable weights and bias values.

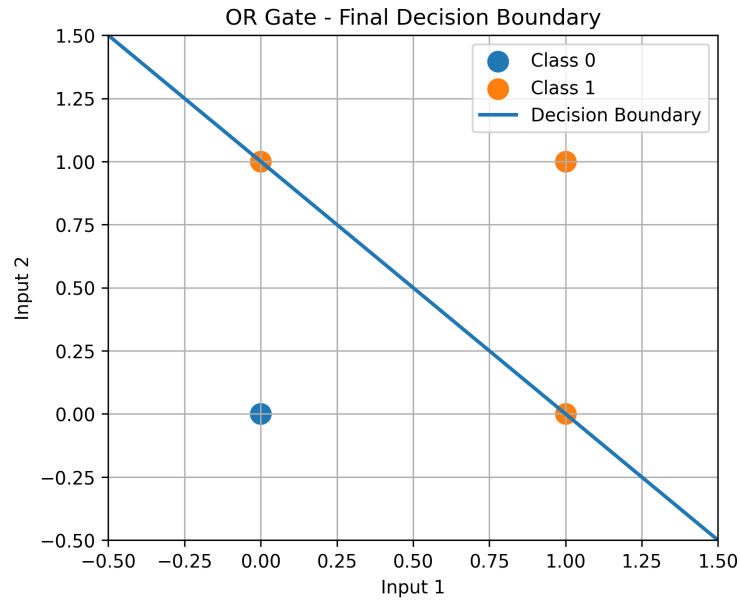


Figure 13: Decision Boundary Learned for OR Gate

The final decision boundary separates the negative sample from the positive samples correctly. This experiment shows that a Single Layer Perceptron is effective for classification problems involving simple linear relationships.

NOT Gate

The NOT gate was implemented using a single input feature. Since there is only one input variable, the decision boundary is represented as a threshold value rather than a two-dimensional separating line.

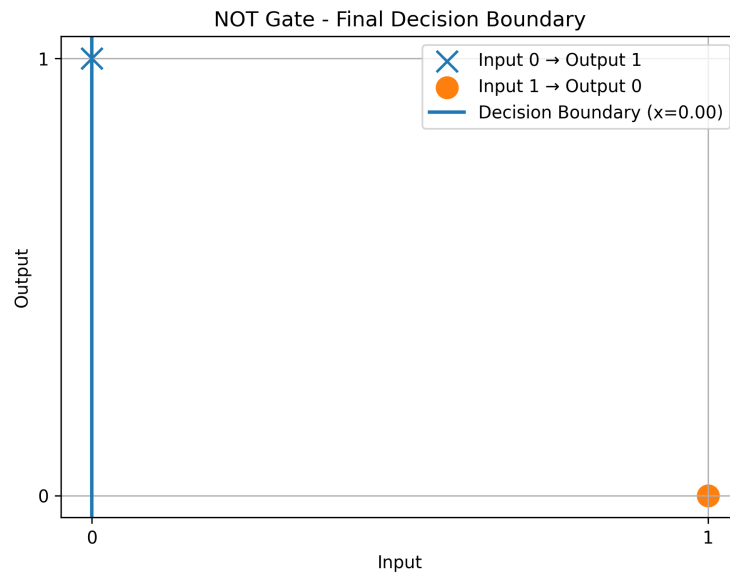


Figure 14: Decision Boundary Learned for NOT Gate

The perceptron successfully learned the inverse relationship between input and output. The learned threshold separates the two possible input values, correctly mapping input 0 to output 1 and input 1 to output 0.

XOR Gate

The XOR gate experiment demonstrates the limitation of a Single Layer Perceptron. Unlike AND and OR gates, XOR is not linearly separable because the samples belonging to the same class are positioned diagonally opposite to each other.

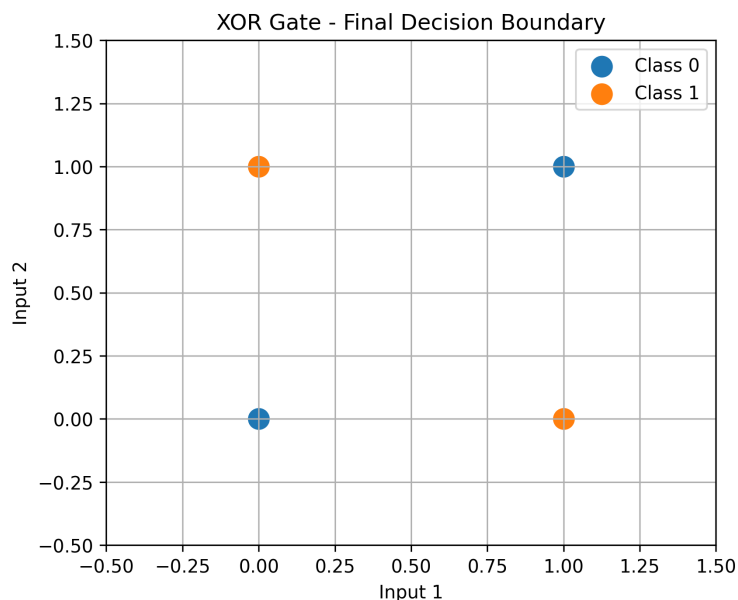


Figure 15: Attempted Decision Boundary for XOR Gate

The perceptron was unable to converge for the XOR problem because no single straight line can correctly separate the two classes. The weights continued to update without reaching a stable solution. This limitation motivates the use of Multi Layer Perceptrons with hidden layers and nonlinear activation functions.

11. Discussion

The perceptron achieved very high accuracy (98.91%) on the Banknote Authentication dataset, which is expected given the near-linear separability of the classes, clearly visible in the scatter plot and correlation heatmap. The training error never fully converged to zero, oscillating instead in a narrow band – this indicates a small number of borderline samples are not perfectly linearly separable, a known limitation of the perceptron learning rule. Comparison with Scikit-learn’s implementation highlighted the effect of default hyperparameters (early stopping, shuffling) on final performance, even when both models use the same underlying learning rule.

12. Conclusion

A Single Layer Perceptron was successfully implemented from scratch and evaluated on the Banknote Authentication dataset, achieving 98.91% accuracy and an F1-score of 0.9878. The experiment demonstrated the perceptron's effectiveness on linearly separable data, while also revealing its core limitations – non-convergence on borderline samples and inability to solve non-linearly-separable problems such as XOR – motivating the need for multilayer architectures studied in subsequent experiments.

13. References

GitHub Repository

The complete implementation code, experiment notebooks, reports, and generated plots are available in the following GitHub repository:

<https://github.com/D-DRM/Deep-Learning-Lab>

1. UCI Machine Learning Repository – Banknote Authentication Dataset.
2. Scikit-learn Documentation: Perceptron.